Programmation Frontend

Lab 2 — Composants, Props et CSS Modules

Organiser et styliser une application React

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Avoir terminé le Lab 1 (dossier `annuaire-react/` fonctionnel).
Fichier de départ	Le fichier `src/App.jsx` du Lab 1
Livrables	3 composants (`App`, `UserCard`, `SearchBar`) avec CSS Modules
Validation	Le filtre en temps réel fonctionne, chaque composant a son propre fichier

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. **Découper** une application React en composants réutilisables
2. **Passer des données** d'un composant parent à un composant enfant via les **props**
3. **Styliser** chaque composant avec un **CSS Module** (pas de collision de noms)
4. **Remonter l'état** vers le parent commun (*lifting state up*)
5. Implémenter un **filtre en temps réel** sur une liste

Table des matières

1 Point de départ — Le code du Lab 1 **3**
1.1 Le problème 4

2 Partie 1 — Extraire un composant **4**
2.1 Créer le dossier `components/` 4
2.2 Créer `UserCard.jsx` 4
2.3 Utiliser `UserCard` dans `App.jsx` 5

3 Partie 2 — Les props **6**
3.1 Comment les données circulent . . . 6
3.2 Props multiples . . . 6

4 Partie 3 — CSS Modules **7**
4.1 Le problème des styles en ligne . . . 7
4.2 CSS Modules : du CSS scopé par composant . . . 7
4.3 Utiliser le CSS Module dans le composant . . . 8
4.4 CSS Module pour App . . . 8

5 Partie 4 — Créer le composant SearchBar **10**
5.1 Où placer la recherche ? . . . 10
5.2 Créer `SearchBar.module.css` . . . 10
5.3 Créer `SearchBar.jsx` . . . 11
5.4 Connecter `SearchBar` dans `App.jsx` . . . 11
5.5 Tester le filtre . . . 13

6 Bilan — Structure du projet **13**

7 Aide-mémoire **14**

8 Résumé et conclusion **15**

1 Point de départ — Le code du Lab 1

Au Lab 1, vous avez créé une application React qui récupère des utilisateurs depuis JSONPlaceholder et les affiche en cartes. Tout fonctionne. Mais regardez le fichier `src/App.jsx` :

```
import { useState } from "react";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  return (
    <div style={{ maxWidth: "600px", margin: "2rem auto",
                  fontFamily: "sans-serif" }}>
      <h1>Annuaire des utilisateurs</h1>
      <button onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger les utilisateurs"}
      </button>

      {users.map(user => (
        <div key={user.id} style={{ border: "1px solid #ccc",
             padding: "1rem", margin: "0.5rem 0",
             borderRadius: "8px" }}>
          <h3 style={{ margin: "0 0 0.5rem 0" }}>{user.name}</h3>
          <p style={{ margin: "0.2rem 0", color: "#555" }}>
            Email : {user.email}
          </p>
          <p style={{ margin: "0.2rem 0", color: "#555" }}>
            Entreprise : {user.company.name}
          </p>
        </div>
      ))}
    </div>
  );
};

export default App;
```

Étape 0 — Vérification

Ouvrez le dossier `annuaire-react/` dans VS Code. Lancez le serveur de développement :

```
cd annuaire-react
npm run dev
```

Ouvrez `http://localhost:5173`. Cliquez sur le bouton. Les 10 utilisateurs doivent apparaître en cartes.
Si le projet ne démarre pas, vérifiez que les dépendances sont installées (`npm install`).

1.1 Le problème

Le code fonctionne, mais tout est dans un seul fichier :

- La logique de chargement (fetch, loading)
- L'affichage de chaque carte utilisateur (le bloc `<div>` dans le `.map()`)
- Les styles (en `style={{...}}` directement dans le JSX)

Imaginez que Fatou doit ajouter un champ de recherche, que Moussa doit modifier l'apparence des cartes, et que Cheikh doit ajouter un compteur. Les trois travaillent dans le **même fichier**. Ça ne tient pas.

La solution : **découper en composants**. Chaque morceau d'interface devient un fichier indépendant avec sa propre logique et son propre style.

2 Partie 1 — Extraire un composant

Objectif — 15 minutes

Extraire la carte utilisateur dans un composant `UserCard` séparé.

2.1 Créer le dossier `components/`

Dans `src/`, créez un dossier `components/` :

```
mkdir src/components
```

Ce dossier contiendra tous les composants réutilisables de l'application.

2.2 Créer `UserCard.jsx`

Créez le fichier `src/components/UserCard.jsx` :

```
const UserCard = ({ user }) => {
  return (
    <div style={{ border: "1px solid #ccc", padding: "1rem",
                  margin: "0.5rem 0", borderRadius: "8px" }}>
      <h3 style={{ margin: "0 0 0.5rem 0" }}>{user.name}</h3>
      <p style={{ margin: "0.2rem 0", color: "#555" }}>
        Email : {user.email}
      </p>
      <p style={{ margin: "0.2rem 0", color: "#555" }}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
```

```
};

export default UserCard;
```

On a simplement **coupé** le bloc `<div>` qui était dans le `.map()` et on l'a mis dans sa propre fonction. Le composant reçoit un objet `user` et affiche ses informations.

2.3 Utiliser `UserCard` dans `App.jsx`

Modifiez `src/App.jsx` pour importer et utiliser le nouveau composant :

```
import { useState } from "react";
import UserCard from "./components/UserCard";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  return (
    <div style={{ maxWidth: "600px", margin: "2rem auto",
                  fontFamily: "sans-serif" }}>
      <h1>Annuaire des utilisateurs</h1>
      <button onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger les utilisateurs"}
      </button>

      {users.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
};

export default App;
```

Sauvegardez les deux fichiers. La page se met à jour automatiquement. Le résultat est **identique** à avant, mais le code est mieux organisé.

Vérification

Si vous voyez une erreur du type `UserCard is not defined` : vérifiez que l'import est correct. Le chemin est relatif : `"./components/UserCard"` (avec le `./` au début).

3 Partie 2 — Les props

Objectif — 15 minutes

Comprendre comment les données circulent entre composants grâce aux props.

3.1 Comment les données circulent

Dans le code qu'on vient d'écrire, regardez cette ligne dans `App.jsx` :

```
<UserCard key={user.id} user={user} />
```

Et cette ligne dans `UserCard.jsx` :

```
const UserCard = ({ user }) => {
```

Le mot `user` apparaît des deux côtés. Voici ce qui se passe :

1. `App` **envoie** l'objet `user` au composant `UserCard` via l'attribut `user={user}`.
2. `UserCard` **reçoit** cet objet dans ses **props** (propriétés).
3. L'écriture `({ user })` est une **déstructuration** : on extrait directement la propriété `user` de l'objet props.

Analogie : la fiche de commande

Imaginez un restaurant. Le serveur (le composant parent) remplit une fiche de commande et la passe au cuisinier (le composant enfant). La fiche contient les informations nécessaires : « 1 thiéboudienne, pas trop pimenté ». Le cuisinier lit la fiche et prépare le plat. Il ne modifie **pas** la fiche — il la lit seulement.
Les props fonctionnent pareil : le parent écrit, l'enfant lit. **Les props sont en lecture seule.**

3.2 Props multiples

Un composant peut recevoir autant de props que nécessaire. Modifions `UserCard` pour recevoir une prop supplémentaire qui contrôle l'affichage de l'email :

```
const UserCard = ({ user, afficherEmail }) => {
  return (
    <div style={{ border: "1px solid #ccc", padding: "1rem",
                  margin: "0.5rem 0", borderRadius: "8px" }}>
      <h3 style={{ margin: "0 0 0.5rem 0" }}>{user.name}</h3>
      {afficherEmail && (
        <p style={{ margin: "0.2rem 0", color: "#555" }}>
          Email : {user.email}
        </p>
      )}
      <p style={{ margin: "0.2rem 0", color: "#555" }}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
};

export default UserCard;
```

Dans `App.jsx` :

```
<UserCard key={user.id} user={user} afficherEmail={true} />
```

La syntaxe `{afficherEmail && (...)}` signifie : « si `afficherEmail` est vrai, affiche ce bloc ». C'est du **rendu conditionnel**.

Les props en résumé

Concept	Explication
Passer une prop	`<Composant nom={valeur} />`
Recevoir une prop	`const Composant = ({ nom }) => { ... }`
Props multiples	`<Composant a={1} b={"texte"} c={true} />`
Lecture seule	L'enfant ne modifie **jamais** les props reçues
`key`	Identifiant unique dans un `.map()` (pas une prop classique)

Pour la suite du lab, retirez la prop `afficherEmail` et revenez à la version simple de `UserCard` (qui affiche toujours l'email). On ajoute cette prop uniquement pour illustrer le concept.

4 Partie 3 — CSS Modules

Objectif — 15 minutes

Remplacer les styles en ligne par des CSS Modules pour une meilleure organisation.

4.1 Le problème des styles en ligne

Dans le code actuel, les styles sont écrits directement dans le JSX :

```
<div style={{ border: "1px solid #ccc", padding: "1rem",
              margin: "0.5rem 0", borderRadius: "8px" }}>
```

Trois problèmes :

- Le code est **difficile à lire** (mélange de HTML et de CSS).
- On ne peut pas utiliser les **pseudo-classes** (`:hover`, `:focus`).
- Si deux composants utilisent la même classe CSS, il y a un risque de **collision de noms**.

4.2 CSS Modules : du CSS scopé par composant

Un **CSS Module** est un fichier CSS classique dont les noms de classes sont **automatiquement rendus uniques** par Vite. Chaque composant a son propre fichier `.module.css`. Pas de collision possible.

Créez le fichier `src/components/UserCard.module.css` :

```
.card {
  border: 1px solid #ccc;
  padding: 1rem;
  margin: 0.5rem 0;
  border-radius: 8px;
  transition: box-shadow 0.2s;
}
```

```
.card:hover {
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.nom {
  margin: 0 0 0.5rem 0;
}

.info {
  margin: 0.2rem 0;
  color: #555;
}
```

Notez le `:hover` : on peut maintenant utiliser les pseudo-classes, ce qui était impossible avec les styles en ligne.

4.3 Utiliser le CSS Module dans le composant

Modifiez `src/components/UserCard.jsx` :

```
import styles from "./UserCard.module.css";

const UserCard = ({ user }) => {
  return (
    <div className={styles.card}>
      <h3 className={styles.nom}>{user.name}</h3>
      <p className={styles.info}>Email : {user.email}</p>
      <p className={styles.info}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
};

export default UserCard;
```

Comment fonctionne un CSS Module

1. Vous écrivez `.card` dans le fichier CSS.
2. Vite le transforme en quelque chose comme `.UserCard_card_x7k2q` (un nom unique).
3. Vous y accédez avec `styles.card` dans le JSX.
4. Résultat : même si un autre composant a aussi une classe `.card`, il n'y a **aucune collision**.

Notez aussi : en React, on écrit `className` au lieu de `class` (car `class` est un mot réservé en JavaScript).

4.4 CSS Module pour App

Créez `src/App.module.css` :

```
.container {
  max-width: 600px;
  margin: 2rem auto;
  font-family: sans-serif;
  padding: 0 1rem;
```

```
}

.titre {
  color: #333;
}

.bouton {
  padding: 0.5rem 1.5rem;
  font-size: 1rem;
  cursor: pointer;
  border: 1px solid #ccc;
  border-radius: 6px;
  background: #f9f9f9;
  transition: background 0.2s;
}

.bouton:hover {
  background: #eee;
}

.bouton:disabled {
  cursor: not-allowed;
  opacity: 0.6;
}
```

Modifiez `src/App.jsx` :

```
import { useState } from "react";
import UserCard from "./components/UserCard";
import styles from "./App.module.css";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  return (
    <div className={styles.container}>
      <h1 className={styles.titre}>Annuaire des utilisateurs</h1>
      <button className={styles.bouton}
              onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger les utilisateurs"}
      </button>
```

```
      {users.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
};

export default App;
```

Sauvegardez et vérifiez dans le navigateur. L'application fonctionne comme avant, avec en plus un effet `:hover` sur les cartes et sur le bouton.

Avant / après : les styles

	Styles en ligne (Lab 1)	**CSS Modules** (Lab 2)
Syntaxe	`style={{ color: "red" }}`	`className={styles.nom}`
Pseudo-classes	Impossible	`:hover`, `:focus`, `:disabled`
Collision	Aucun risque (mais pas pratique)	Aucun risque (noms rendus uniques)
Lisibilité	Mélange JS + CSS	Séparation claire

5 Partie 4 — Créer le composant SearchBar

Objectif — 20 minutes

Créer un composant de recherche et le connecter à la liste pour filtrer les utilisateurs en temps réel.

5.1 Où placer la recherche ?

On veut qu'en tapant dans un champ de recherche, la liste se filtre instantanément. Pour cela, il faut que :

1. Le composant `SearchBar` connaisse le texte tapé par l'utilisateur.
2. Le composant `App` connaisse aussi ce texte pour filtrer la liste.

Le texte de recherche est donc un **état partagé** entre `SearchBar` et `App`. En React, quand deux composants ont besoin du même état, on le place dans leur **parent commun**. C'est le principe du *lifting state up* (remonter l'état).

Analogie : le tableau de la classe

Imaginez que Fatou et Moussa doivent consulter la même information. Au lieu que chacun ait sa propre copie (qui risque de diverger), on écrit l'information au tableau — visible par tous. Le tableau, c'est le composant parent. L'information, c'est l'état.

5.2 Créer `SearchBar.module.css`

Créez `src/components/SearchBar.module.css` :

```
.container {
  margin: 1rem 0;
}
```

```css
.input {
  width: 100%;
  padding: 0.6rem 1rem;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 6px;
  box-sizing: border-box;
}

.input:focus {
  outline: none;
  border-color: #666;
}
```

5.3 Créer `SearchBar.jsx`

Créez `src/components/SearchBar.jsx` :

```jsx
import styles from "./SearchBar.module.css";

const SearchBar = ({ recherche, onRecherche }) => {
  return (
    <div className={styles.container}>
      <input
        className={styles.input}
        type="text"
        placeholder="Rechercher un utilisateur..."
        value={recherche}
        onChange={e => onRecherche(e.target.value)}
      />
    </div>
  );
};

export default SearchBar;
```

Le composant reçoit **deux props** :

- `recherche` : la valeur actuelle du champ (vient du parent).
- `onRecherche` : une **fonction** que le parent fournit. Quand l'utilisateur tape, on appelle cette fonction avec le nouveau texte.

Passer une fonction comme prop

En React, on passe souvent des fonctions comme props. Ici, `onRecherche` est une fonction fournie par `App`. Quand `SearchBar` appelle `onRecherche("Fatou")`, c'est en réalité `App` qui reçoit `"Fatou"` et met à jour son état. Le composant enfant **informe** le parent, sans modifier l'état directement.

5.4 Connecter `SearchBar` dans `App.jsx`

Modifiez `src/App.jsx` :

```jsx
import { useState } from "react";
import UserCard from "./components/UserCard";
import SearchBar from "./components/SearchBar";
import styles from "./App.module.css";
```

```
const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);
  const [recherche, setRecherche] = useState("");

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  const usersFiltres = users.filter(user =>
    user.name.toLowerCase().includes(
      recherche.toLowerCase()
    )
  );

  return (
    <div className={styles.container}>
      <h1 className={styles.titre}>Annuaire des utilisateurs</h1>
      <button className={styles.bouton}
              onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger les utilisateurs"}
      </button>

      <SearchBar
        recherche={recherche}
        onRecherche={setRecherche}
      />

      <p className={styles.compteur}>
        {usersFiltres.length} utilisateur(s) trouvé(s)
      </p>

      {usersFiltres.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
};

export default App;
```

Ajoutez la classe `.compteur` dans `src/App.module.css` :

```
.compteur {
  color: #888;
  font-size: 0.9rem;
```

```
  margin: 0.5rem 0;
}
```

5.5 Tester le filtre

Sauvegardez tout. Chargez les utilisateurs, puis tapez dans le champ de recherche. La liste se filtre **en temps réel** à chaque lettre tapée. Le compteur indique le nombre de résultats.

Flux de données du filtre

Voici comment les données circulent dans l'application :

1. L'utilisateur tape « lea » dans `SearchBar`.
2. `SearchBar` appelle `onRecherche("lea")`, qui est en réalité `setRecherche("lea")` dans `App`.
3. L'état `recherche` dans `App` passe à `"lea"`.
4. React re-rend `App`. Le filtre `users.filter(...)` ne garde que les utilisateurs dont le nom contient « lea ».
5. Le tableau filtré est passé au `.map()`, qui ne crée des `UserCard` que pour les résultats.

Le flux est **unidirectionnel** : les données descendent (parent → enfant via props), les événements remontent (enfant → parent via fonctions callback).

6 Bilan — Structure du projet

Votre projet a maintenant cette structure :

```
annuaire-react/
  src/
    App.jsx
    App.module.css
    components/
      UserCard.jsx
      UserCard.module.css
      SearchBar.jsx
      SearchBar.module.css
    main.jsx
  index.html
  package.json
```

Chaque composant a **deux fichiers** : un `.jsx` pour la logique et un `.module.css` pour le style. Si Fatou veut modifier l'apparence des cartes, elle ne touche que `UserCard.module.css`. Si Moussa veut changer le comportement de la recherche, il ne touche que `SearchBar.jsx`. Personne ne se marche dessus.

Avant / après : la structure

	Lab 1	Lab 2
Fichiers	1 fichier (`App.jsx`)	6 fichiers (3 composants × 2)
Styles	En ligne (`style={{...}}`)	CSS Modules (`.module.css`)
Recherche	Aucune	Filtre en temps réel
Réutilisabilité	Aucune	`UserCard` réutilisable partout
Collaboration	Difficile (1 fichier)	Facile (fichiers séparés)

7 Aide-mémoire

Concept	**Syntaxe / Exemple**
Créer un composant	`const MonComposant = () => <div>...</div>;`
Exporter	`export default MonComposant;`
Importer	`import MonComposant from "./MonComposant";`
Passer une prop	`<Enfant nom={valeur} />`
Recevoir une prop	`const Enfant = ({ nom }) => ...`
Prop fonction	`<Enfant onAction={maFonction} />`
Rendu conditionnel	`{condition && <Element />}`
CSS Module (fichier)	`MonComposant.module.css`
CSS Module (import)	`import styles from "./MonComposant.module.css";`
CSS Module (usage)	`className={styles.maClasse}`
Filtrer un tableau	`tableau.filter(item => condition)`
Lifting state up	L'état partagé vit dans le parent commun

8 Résumé et conclusion

Dans ce lab, vous avez :

1. **Extrait** la carte utilisateur dans un composant `UserCard` réutilisable.
2. Compris les **props** : des données en lecture seule passées du parent vers l'enfant.
3. Remplacé les styles en ligne par des **CSS Modules** (un fichier `.module.css` par composant).
4. Créé un composant `SearchBar` et implémenté un **filtre en temps réel**.
5. Appliqué le *lifting state up* : l'état de recherche vit dans `App` et descend vers `SearchBar` et la liste filtrée.

Voici où vous en êtes dans la progression du cours :

Étape	Statut	Ce que vous savez faire
DOM et événements	✓	Sélecteurs, événements, modification du DOM
Fonctions JS	✓	Déclaration, expression, fléchée, *hoisting*
Asynchrone	✓	`fetch`, `async/await`, gestion d'erreurs
React — les bases	✓	Composant, JSX, `useState`, `.map()`
Composants et props	✓	Découpage, props, CSS Modules, lifting state up
useEffect et cycle de vie	→ Lab 3	Charger au montage, loading, détail
Formulaires et routing	→ Lab 4	Inputs contrôlés, React Router

Et ensuite ?

L'annuaire affiche des utilisateurs et permet de les filtrer. Mais il faut encore cliquer sur un bouton pour charger les données. Dans le **Lab 3**, vous apprendrez `useEffect` pour charger les données **automatiquement** au démarrage, gérer les états de chargement et d'erreur proprement, et créer une page de détail pour chaque utilisateur.

Exercice bonus — Pour aller plus loin

Bonus : un composant Header

Si vous avez terminé en avance, créez un composant `Header.jsx` + `Header.module.css` qui contient le titre de l'application et le bouton de chargement. Indices :

1. Créez `src/components/Header.jsx` et `src/components/Header.module.css`.
2. Le composant `Header` reçoit trois props : `onCharger` (la fonction de chargement), `chargement` (booléen), et `nbResultats` (nombre).
3. Dans `App.jsx`, remplacez le `<h1>` et le `<button>` par `<Header />`.
4. L'application doit fonctionner exactement comme avant.

Cet exercice renforce le découpage en composants et les props (y compris passer des fonctions).

Lab 2 — Composants, Props et CSS Modules — Terminé ✓

Vous savez découper une application React en composants,
passer des données avec les props et styliser avec CSS Modules.

Prochaine étape : `useEffect`, chargement automatique et page de détail.

Un composant = une responsabilité. Un fichier = un composant.